



ZAP by
Checkmarx

ZAP by Checkmarx Scanning Report

Site: <http://127.0.0.1:3000>

Generated on Mon, 2 Mar 2026 14:24:42

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	3
Low	3
Informational	3

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP errors logged - see the zap. log file for details	12
Low	Warning		ZAP warnings logged - see the zap. log file for details	2
Info	Informational		Percentage of network failures	8 %
Info	Informational	http://127.0.0.1:3000	Percentage of responses with status code 2xx	28 %
Info	Informational	http://127.0.0.1:3000	Percentage of responses with status code 3xx	68 %
Info	Informational	http://127.0.0.1:3000	Percentage of responses with status code 4xx	1 %

Info	Informational	http://127.0.0.1:3000	Percentage of responses with status code 5xx	1 %
Info	Informational	http://127.0.0.1:3000	Percentage of endpoints with content type image/png	10 %
Info	Informational	http://127.0.0.1:3000	Percentage of endpoints with content type text/html	80 %
Info	Informational	http://127.0.0.1:3000	Percentage of endpoints with content type text/javascript	10 %
Info	Informational	http://127.0.0.1:3000	Percentage of endpoints with method GET	70 %
Info	Informational	http://127.0.0.1:3000	Percentage of endpoints with method POST	30 %
Info	Informational	http://127.0.0.1:3000	Count of total endpoints	10
Info	Informational	http://127.0.0.1:3000	Percentage of slow responses	4 %

Alerts

Name	Risk Level	Number of Instances
Absence of Anti-CSRF Tokens	Medium	3
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Missing Anti-clickjacking Header	Medium	4
Application Error Disclosure	Low	2
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Authentication Request Identified	Informational	1

Information Disclosure - Suspicious Comments	Informational	1
Modern Web Application	Informational	2

Alert Detail

Medium	Absence of Anti-CSRF Tokens
Description	No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL /form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including: * The victim has an active session on the target site. * The victim is authenticated via HTTP auth on the target site. * The victim is on the same local network as the target site. CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login
Method	GET
Attack	
Evidence	<form method="POST">
Other Info	No known Anti-CSRF token [anticsrf, CSRFTOKEN, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "password" "username"].
URL	http://127.0.0.1:3000/register
Node Name	http://127.0.0.1:3000/register
Method	GET
Attack	
Evidence	<form method="POST" id="registerForm">
Other Info	No known Anti-CSRF token [anticsrf, CSRFTOKEN, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "email" "password" "roleField" "username"].
URL	http://127.0.0.1:3000/register?btn=Confirm+Pin&pin=ZAP
Node Name	http://127.0.0.1:3000/register (btn,pin)
Method	GET

Attack	
Evidence	<form method="POST" id="registerForm">
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "email" "password" "roleField" "username"].
Instances	3
Solution	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script. Phase: Architecture and Design Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Note that this can be bypassed using XSS. Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation. Note that this can be bypassed using XSS. Use the ESAPI Session Management control. This control includes a component for CSRF. Do not use the GET method for any request that triggers a state change. Phase: Implementation Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html
CWE Id	352
WASC Id	9
Plugin Id	10202

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login

Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:3000/register
Node Name	http://127.0.0.1:3000/register
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:3000/register?btn=Confirm+Pin&pin=ZAP
Node Name	http://127.0.0.1:3000/register (btn,pin)
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login ()(password,username)
Method	POST
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:3000/register
Node Name	http://127.0.0.1:3000/register ()(email,password,role,username)
Method	POST
Attack	
Evidence	
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/

CWE Id	693
WASC Id	15
Plugin Id	10038

Medium	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:3000/register
Node Name	http://127.0.0.1:3000/register
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:3000/register?btn=Confirm+Pin&pin=ZAP
Node Name	http://127.0.0.1:3000/register (btn,pin)
Method	GET
Attack	
Evidence	
Other Info	
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login ()(password,username)
Method	POST
Attack	
Evidence	
Other Info	
Instances	4
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

CWE Id	1021
WASC Id	15
Plugin Id	10020

Low	Application Error Disclosure
Description	This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.
URL	http://127.0.0.1:3000/register
Node Name	http://127.0.0.1:3000/register ()(email,password,role,username)
Method	POST
Attack	
Evidence	HTTP/1.1 500 INTERNAL SERVER ERROR
Other Info	
URL	http://127.0.0.1:3000/register?btn=Confirm+Pin&pin=ZAP
Node Name	http://127.0.0.1:3000/register (btn,pin)(email,password,role,username)
Method	POST
Attack	
Evidence	HTTP/1.1 500 INTERNAL SERVER ERROR
Other Info	
Instances	2
Solution	Review the source code of this page. Implement custom error pages. Consider implementing a mechanism to provide a unique error reference/identifier to the client (browser) while logging the details on the server side and not exposing them to the user.
Reference	
CWE Id	550
WASC Id	13
Plugin Id	90022

Low	Server Leaks Version Information via "Server" HTTP Response Header Field
Description	The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login
Method	GET
Attack	
Evidence	Werkzeug/3.0.1 Python/3.14.2
Other Info	
URL	http://127.0.0.1:3000/robots.txt
Node	

Name	http://127.0.0.1:3000/robots.txt
Method	GET
Attack	
Evidence	Werkzeug/3.0.1 Python/3.14.2
Other Info	
URL	http://127.0.0.1:3000/sitemap.xml
Node Name	http://127.0.0.1:3000/sitemap.xml
Method	GET
Attack	
Evidence	Werkzeug/3.0.1 Python/3.14.2
Other Info	
URL	http://127.0.0.1:3000/static/NGS-logo.png
Node Name	http://127.0.0.1:3000/static/NGS-logo.png
Method	GET
Attack	
Evidence	Werkzeug/3.0.1 Python/3.14.2
Other Info	
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login ()(password,username)
Method	POST
Attack	
Evidence	Werkzeug/3.0.1 Python/3.14.2
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.
Reference	https://httpd.apache.org/docs/current/mod/core.html#servertokens https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10036

Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://127.0.0.1:3000/login

Node Name	http://127.0.0.1:3000/login
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:3000/register
Node Name	http://127.0.0.1:3000/register
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:3000/register?__debugger__=yes&cmd=resource&f=debugger.js
Node Name	http://127.0.0.1:3000/register (__debugger__,cmd,f)
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:3000/static/NGS-logo.png
Node Name	http://127.0.0.1:3000/static/NGS-logo.png
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login ()(password,username)
Method	POST
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security-Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	Authentication Request Identified
Description	The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.
URL	http://127.0.0.1:3000/login
Node Name	http://127.0.0.1:3000/login ()(password,username)
Method	POST
Attack	
Evidence	password
Other Info	userParam=username userValue=ZAP passwordParam=password referer=http://127.0.0.1:3000/login
Instances	1
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/
CWE Id	
WASC Id	
Plugin Id	10111

Informational	Information Disclosure - Suspicious Comments
Description	The response appears to contain suspicious comments which may help an attacker.
URL	http://127.0.0.1:3000/register?__debugger__=yes&cmd=resource&f=debugger.js
Node Name	http://127.0.0.1:3000/register (__debugger__,cmd,f)
Method	GET
Attack	
Evidence	from
Other Info	The following pattern was used: \bFROM\b and was detected in likely comment: "// Prevent page from refreshing.", see evidence field for the suspicious comment/snippet.
Instances	1
Solution	Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.
Reference	
CWE Id	615

WASC Id	13
Plugin Id	10027
Informational	Modern Web Application
Description	The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.
URL	http://127.0.0.1:3000/register
Node Name	http://127.0.0.1:3000/register()(email,password,role,username)
Method	POST
Attack	
Evidence	<script src="__debugger__=yes&cmd=resource&f=debugger.js"></script>
Other Info	No links have been found while there are scripts, which is an indication that this is a modern web application.
URL	http://127.0.0.1:3000/register?btn=Confirm+Pin&pin=ZAP
Node Name	http://127.0.0.1:3000/register(btn,pin)(email,password,role,username)
Method	POST
Attack	
Evidence	<script src="__debugger__=yes&cmd=resource&f=debugger.js"></script>
Other Info	No links have been found while there are scripts, which is an indication that this is a modern web application.
Instances	2
Solution	This is an informational alert and so no changes are required.
Reference	
CWE Id	
WASC Id	
Plugin Id	10109